RĪGAS TEHNISKĀ UNIVERSITĀTE

Datorzinātnes, informācijas tehnoloģijas un enerģētikas fakultāte

Informācijas tehnoloģijas institūts

Rihards Ieviņš

Akadēmisko maģistra (otrā cikla) studiju programmas

Informācijas tehnoloģija

students, stud. apl. nr. 211RDB309

API IZSTRĀDES UN DOKUMENTĒŠANAS AUTOMATIZĀCIJAS RISINĀJUMA IZSTRĀDE AR ĢENERATĪVO MĀKSLĪGO INTELEKTU

MAĢISTRA DARBS

Zinātniskā vadītāja Dr. sc. Ing. profesore

Oksana Ņikiforova

RĪGA 2025

RĪGAS TEHNISKĀ UNIVERSITĀTE

DATORZINĀTNES, INFORMĀCIJAS TEHNOLOĢIJAS UN ENERĢĒTIKAS FAKULTĀTE

Informācijas tehnoloģijas institūts

Maģistra darba izpildes lapa

Noslēguma darba autors:

students Rihards Ieviņš _____

(paraksts, datums)

Noslēguma darbs ieteikts aizstāvēšanai:

Zinātniskā vadītāja:

Dr. sc. ing. profesore Oksana Ņikiforova _____

(paraksts, datums)

ANOTĀCIJA

API SPECIFIKĀCIJA, ĢENERATĪVAIS MĀKSLĪGAIS INTELEKTS, PROGRAMMINŽENIERIJA, LIELIE VALODAS MODEĻI, OPENAPI, API DOKUMENTĀCIJA, API DOKUMENTĀCIJAS AUTOMATIZĀCIJA

API izmantošana ir būtisks elements mūsdienu programmatūras sistēmu integrācijā un savstarpējā komunikācijā. API dokumentācijas kvalitāte ietekmē programmatūru vai sistēmu uzturējamību, atkārtotu izmantošanu un integrācijas efektivitāti. Manuāla API specifikāciju izstrāde ir resurss-ietilpīgs process, kas var veidot nekonsekvenci starp faktisko API funkcionalitāti un tā dokumentācijas saturu. Maģistra darba mērķis ir izstrādāt metodi un demonstrācijas rīku API specifikāciju un dokumentācijas automatizētai ģenerēšanai, izmantojot ģeneratīvo mākslīgo intelektu. Darbā tiek analizēti API izstrādes principi, *OpenAPI* standarts un esošie dokumentācijas rīki. Tiek izvērtētas lielo valodas modeļu iespējas strukturēta API specifikāciju ģenerēšanā, ja ir doti sistēmu prasības teksti vai programmatūras koda fragmenti jeb daļas. Darba ietvaros izstrādāts datu plūsmas modelis un realizēts demonstrācijas prototips, kas nodrošina API galapunktu, parametru un datu struktūru identificēšanu un to izstrādi atbilstoši standartiem. Veikta salīdzinošā analīze starp prototipa ģenerēto un manuāli izstrādāto dokumentāciju, novērtējot precizitāti, struktūras pareizību un atbilstību standartam. Izstrādātais risinājums piedāvāts kā pamats API dokumentācijas automatizācijas turpmākai attīstībai programmatūras izstrādes kontekstā.

Darba apjoms - X lpp., X attēli, 1 tabulas, XX informācijas avoti un X pielikumi.

ABSTRACT

API SPECIFICATION, GENERATIVE ARTIFICIAL INTELLIGENCE, SOFTWARE ENGINEERING, LARGE LANGUAGE MODELS, OPENAPI, API DOCUMENTATION, API DOCUMENTATION AUTOMATION

Master's thesis title: Development of a Solution for API Design and Documentation Automation with Generative Artificial Intelligence.

The master's thesis consists of X pages, X images, 1 tables, X information sources and X attachments.

SATURA RĀDĪTĀJS

IEVADS 3
1. API IZSTRĀDES UN DOKUMENTĒŠANAS TEORĒTISKIE PAMATI 6
1.1. API un OpenAPI standarta jēdziens un loma sistēmu integrācijā 6
1.2. RESTful un GraphQL API salīdzinājums 7
1.3. Esošie API dokumentācijas rīki un to ierobežojumi 10
1.3.1. API dokumentācijas rīku loģika 10
1.3.2. API dokumentācijas rīku piemēri 11
1.3.3. Ģeneratīvā mākslīgā intelekta bāzēti API dokumentācijas rīki 13
1.4. API dokumentācijas kvalitātes kritēriji 14
2. MĀKSLĪGĀ INTELEKTA IZMANTOŠNA API ĢENERĒŠANĀ 16
2.1. Ģeneratīvo modeļu uzbūve un to loma dabīgās valodas pārveidē strukturētā formā 16
2.2. LLM pielietojumi programmatūras inženierijā 18
2.3. Prompt Engineering un datu konteksta struktūras API ģenerēšanai 20
2.4. Izaicinājumi - semantiskā precizitāte, drošība, versiju kontrole 22
3. AI BALSTĪTAS API SPECIFIKĀCIJAS ĢENERĒŠANAS METODES IZSTRĀDE 24
3.1 Metodes arhitektūras un ievades/izvades datu modelis 24
3.2. Prasību teksta un koda fragmentu apstrāde 26
3.3. API galapunktu, parametru un datu tipu identificēšanas mehānisms 28
3.4. AI modeļa rezultātu validācija pret OpenAPI shēmu 30
4. PROTOTIPA IZSTRĀDE UN VALIDĀCIJA 32
4.1. Izmantotā tehnoloģiju kopa 32
4.2. Prototipa funkcionalitāte 32
4.3. Testa scenāriji un datu kopas 32
4.4. Lietotāju testēšana un rezultātu novērtējums 32
5. AI ĢENERĒTĀS UN CILVĒKA VEIDOTĀS DOKUMENTĀCIJAS SALĪDZINOŠĀ ANALĪZE 33
5.1. Novērtēšanas kritēriji 33
5.2. Rezultāti un to analīze 33
5.3. Trūkumi un uzlabojumu iespējas 33
REZULTĀTI UN SECINĀJUMI 34

IZMANTOTIE INFORMĀCIJAS AVOTI 35
PIELIKUMI 46

IEVADS

Programmatūras izstrādē API (*Application Programming Interface*) funkcija ir nozīmīga, jo tā nodrošina esošo komponenšu savstarpēju sadarbību, kā arī sistēmu un pakalpojumu savienojamību un integrāciju. API pieejamības, atkārtotas izmantošanas un uzturēšanas nodrošināšanai ir nepieciešama kvalitatīva API dokumentācija - strukturēta, precīza un atbilst standartiem. Taču API specifikāciju veidošana pieprasa ievērojamu resursu patēriņu, jo bieži vien tā balstās uz manuālu informācijas apkopošanu, strukturēšanu. Šāda pieeja var novest pie nekonsekvences starp faktisko API funkcionalitāti un dokumentācijas saturu. (Decrop et al., 2025)

Mākslīgā intelekta tehnoloģiju attīstība un to izmantošanas veicināšana, it īpaši lielo valodas modeļu (*LLM - Large Language Models*) izmantošanas iespējas, rada iespējas izstrādāt automatizētus risinājumus API specifikāciju izstrādei un dokumentēšanai. Lielo valodas modeļu spējas apstrādāt gan dabiskās valodas tekstu, gan sniegto programmatūras kodu ļauj radīt strukturētus rezultātus, kas atbilst standartiem, piemēram, *OpenAPI* formatējumam. Šāda risinājuma izstrāde potenciāli samazinātu cilvēka manuālo darba apjomu, palielinātu dokumentācijas precizitāti, konsekventumu ar API funkcionalitāti salīdzinājumā ar tradicionālajām metodēm. (Lazar, Vetzler, Kate, Tsay, Gupta, et al., 2025)

Ņemot vērā API dokumentācijas nozīmi programmatūras izstrādē un ar manuālās pieejas saistītos izaicinājumus, ir aktuāli izstrādāt un izvērtēt automatizētu pieeju, kas nodrošinātu gan efektivitāti, gan atbilstību nozares standartiem.

Maģistra darba mērķis ir izstrādāt metodi un demonstrācijas rīku, kas izmantotu ģeneratīvo mākslīgo intelektu priekš API specifikāciju ģenerēšanas un dokumentēšanas, balstoties uz lietotāja dotajiem ievad-datiem - sistēmas prasību tekstiem vai koda fragmentiem.

Darba mērķa sasniegšanai tiek definēti sekojoši uzdevumi:

1) Izpētīt literatūru par API izstrādes principiem, *OpenAPI* standartu un dokumentēšanas rīkiem;
2) Analizēt ģeneratīvā mākslīgā intelekta modeļu (piem., *GPT, Claude, Llama)* piemērotību API struktūras un dokumentācijas ģenerēšanai;
3) Izstrādāt datu plūsmas modeli API specifikācijas ģenerēšanai no dabīgās valodas vai koda;

4) Izstrādāt demonstrācijas prototipu, kas spēj no ievad-datiem (prasību teksta vai koda fragmenta) ģenerēt API specifikāciju un dokumentācijas aprakstu;
5) Veikt salīdzinošu analīzi starp AI ģenerēto un cilvēka veidoto dokumentāciju, novērtējot precizitāti, lasāmību un atbilstību *OpenAPI* standartam;
6) Veikt secinājumus un izstrādāt ieteikumus API dokumentācijas automatizācijai, izmantojot ģeneratīvo mākslīgo intelektu.

Maģistra darba pirmajā daļā tiek aplūkoti API izstrādes un dokumentēšanas teorētiskie pamati. Tiek sniegts vispārīgs pārskats par API jēdzienu un tā nozīmi mūsdienu programmatūras sistēmu integrācijā. Tiek analizēts *OpenAPI* standarts, salīdzināti dažādi API - *RESTful* un *GraphQL*. Tiek izvērtēti arī esošie API dokumentācijas rīki un to ierobežojumi. Kā arī tiek definēti API dokumentācijas kvalitātes kritēriji, kas tiks izmantoti turpmākai analīzei un salīdzinājumiem.

Darba otrajā daļā tiek analizēta mākslīgā intelekta izmantošana API ģenerēšanas kontekstā. Tiek likts akcents uz ģeneratīvo modeļu uzbūvi un to spēju pārveidot dabīgas valodas tekstu strukturētā formā. Tiek aplūkoti lielo valodas modeļu pielietojumi programminženierijā, kā arī analizēta uzvednes inženierijas (*prompt engineering*) nozīme un datu konteksta strukturēšana API ģenerēšanas procesā. Tiek izcelti galvenie izaicinājumi, kas saistīti ar semantisko precizitāti, drošību un API versiju kontroles nodrošināšanu.

Darba trešajā daļā tiek izstrādāta MI balstīta metode API specifikācijas ģenerēšanai, balstoties uz sistēmas prasību tekstiem vai koda fragmentiem. Nodaļā tiek aprakstīta metodes arhitektūra un ievades/izvades datu modelis. Detalizētāk tiek analizēta prasību teksta un koda fragmentu apstrāde, kā arī izstrādāts API galapunktu (*endpointu),* parametru un datu tipu identificēšanas mehānisms. Tiek apskatīta arī AI modeļa ģenerēto rezultātu validācija atbilstoši *OpenAPI* shēmai.

Ceturtajā daļā tiek aprakstīta izstrādātās metodes praktiskā realizācija jau demonstrācijas prototipa veidā. Tiek raksturota izmantotā tehnoloģiju kopa un prototipa funkcionalitāte, kā arī tiek definēti testa scenāriji un tiem izmantotās datu kopas. Papildus tiek veikta prototipa validācija un prototipa lietotāju testēšana, novērtējot izstrādātā risinājuma spēju ģenerēt korektu un izmantojamu API specifikāciju un dokumentāciju.

Piektajā nodaļā tiek veikta salīdzinošā analīze starp AI ģenerēto un cilvēka veidoto API dokumentāciju. Tiek definēti novērtēšanas kritēriji, analīzē iegūtie rezultāti un izvērtēta AI pieejas precizitāte, semantiskā atbildība un struktūras pareizība. Noslēgumā tiek identificēti trūkumi un iespējamās uzlabojumu, attīstības iespējas.

1. API IZSTRĀDES UN DOKUMENTĒŠANAS TEORĒTISKIE PAMATI

Programmatūras inženierijā Lietojumprogrammu saskarnes jeb API (Application Programming Interface) ir būtisks elements, kas nosaka, kā dažādas programmatūras komponentes vai sistēmas savstarpēji mijiedarbojas, apmainās ar informāciju (Casas et al., 2021). API izmantošana ir kļuvusi par pamata arhitektūras daļu mūsdienu sistēmu savstarpējās integrācijās, tostarp REST-orientētās arhitektūrās, kur API dokumentēšana spēj nodrošināt elastīgu, atkārtoti izmantojamu un savietojamu sistēmu vai programmatūras izstrādi (*OpenAPI Specification v3.2.0*, n.d.). Lai panāktu elastīgu statusu, API struktūrai ir jābūt skaidri definētai un konsekventai, kā arī viegli interpretējamai (*OpenAPI Specification v3.2.0*, n.d.).

1.1. API un OpenAPI standarta jēdziens un loma sistēmu integrācijā

API attīstība sākās 20.gadsimta beigās un 21.gadsimta sākumā. Periodā, kad izmantotās programmatūras sistēmas kļuva arvien sarežģītākas jeb komplicētākas un veidojās nepieciešamība pēc standartizētiem mehānismiem, lai komponenti savstarpēji komunicētu efektīvi (Lamothe et al., 2021). API sākotnējā ideja bija nodrošināt strukturētu saskarni, kas ļautu izstrādātājiem izmantot funkcijas vai datus no ārējiem programmatūras komponentiem, bez nepieciešamības izprast to iekšējo implementāciju. Tādejādi veicinot komponentu atkārtotu izmantošanu un modulāras arhitektūras veidošanu (Lamothe et al., 2021).

Mūsdienās API tiek plaši definēts programminženierijas nozares literatūrā kā saskarnes slānis, kas nodrošina savienojamību jeb komunikāciju starp dažādām programmatūras komponentēm vai sistēmām. Tādejādi ļaujot piekļūt funkcionalitātei vai kodam atkārtoti bez nepieciešamības zināt, kādi ir iekšējie procesi. (Boateng et al., 2019) API ne tikai nodrošina datu apmaiņu, bet arī nosaka mijiedarbības nosacījumus starp sistēmām - pieprasījuma struktūru un sagaidāmo atbildi. Šis atvieglo sistēmu integrāciju un programmatūras komponentu savstarpējo sadarbību kompleksās programmatūru izmantošanas arhitektūrās (Boateng et al., 2019).

API nodrošina mehānismu, kas atbalsta programmatūras atkārtotu izmantošanu un veicina modulāru dizainu, jo tas ļauj atkārtoti izmantot funkcionalitāti, tādejādi

samazinot nepieciešamību atkārtoti izstrādāt esošu funkcionalitāti. Šādi arī atvieglojot sistēmu uzturēšanu un paplašināšanu (Irum Raufa et al., 2019).

OpenAPI Specification (OAS) izveide ir saistīta ar nepieciešamību standartizēt REST tipa API aprakstīšanu. Lai spētu nodrošināt vienotu un strukturētu API dokumentēšanas iespēju. Sākotnēji tika izstrādāta *Swagger* specifikācija, kas nodrošināja mašīnlasāmu API aprakstu, kā arī atbalstīja dokumentācijas un rīku izmantošanu REST API izstrādē (Casas et al., 2021). *Swagger* specifikācija tika izmantota API struktūras definēšanai līdz ar ko tas guva plašu pielietojamību API dokumentēšanā un testēšanā. Pēc kā *Swagger* specifikācija tika attīstīta un kļuva par *OpenAPI Specification*, kas kļuva par atvērtu standartu REST API apraksta veidošanai un tiek uzturēta atklātā pārvaldības ietvarā - specifikācijas izmaiņas tiek apspriestas, izstrādātas un apstiprinātas atklātā procesā, kur piedalās vairāki neatkarīgi dalībnieki (industrijas uzņēmumi, rīku izstrādātāji u.c.) (Casas et al., 2021).

OpenAPI specifikācija parasti tiek izteikta JSON vai YAML formātā un ir neatkarīga no kādas konkrētas programmēšanas valoda vai API realizācijas (Casas et al., 2021). OpenAPI apraksts veicina vienkāršāku API struktūras interpretāciju, ko var izmantot automatizētās darbplūsmas, piemēram, dokumentācijas ģenerēšanā, koda izveidē, kā arī API testēšanā un validācijā (Karlsson et al., 2019).

OpenAPI specifikācijas izmantošana nodrošina vienotu un strukturētu informācijas avotu, kas ļauj sistēmām konsekventi interpretēt pieejamo funkcionalitāti (Casas et al., 2021). Šāda standartizēja pieeja samazina interpretācijas kļūdu risku un atvieglo Api izmantošanu integrāciju procesos, jo API ir strukturēts formāli un pēc standarta (Karlsson et al., 2019). OpenAPI specifikāciju ir iespējams izmantot arī testēšanas rīkos, validācijas mehānismos un citos integrācijas atbalsta mehānismos, kas izmanto strukturētu API aprakstu (Karlsson et al., 2019).

1.2. RESTful un GraphQL API salīdzinājums

Mūsdienās ir pieejamas daudzas API arhitektūras metodes, kas dažādos veidos nodrošina strukturētu datu apmaiņu starp sistēmām, programmatūrām un klientiem. Tradicionāli tīmekļa (*web*) pakalpojumu izstrādē dominē resursu orientēti HTTP-balstīti modeļi, kuriem tiek veidoti skaidri galapunkti, kas arī nodrošina gan pieprasījuma, gan atbildes struktūru. Tomēr jaunākas API arhitektūras risinājumi, piemēram, GraphQL, nav tik tradicionāli un piedāvā elastīgāku piekļuves risinājumu -

klienti, klients definē datu apjomu, ko vēlas iegūt. Šāda veida risinājums var atvieglot izstrādes procesu, ja tiek izmantotas kompleksas sistēmas, programmatūras (Surbhi Kanthed, 2023).

RESTful API arhitektūra ir balstīta uz REST (*Representational State Transfer*) principiem. Principi paredz, ka dati tiek strukturēti kā resursi ar savām adresēm, kuriem klients var nosūtīt HTTP pieprasījumus:

- Iegūt datus (GET),
- Izveidot jaunu ierakstu (POST),
- Atjaunināt esošo datu ierakstu vai izveidot jaunu ierakstu (PUT),
- Dzēst nevajadzīgo ierakstu (DELETE).

Arhitektūra nodrošina skaidru, paredzamu un atkārtoti izmantojamu resursu organizēšanu, atbalsta vienkāršu mijiedarbību starp klienta un servera komponentēm, kā arī ir pieejams ievērojams rīku un tehnoloģiju atbalsts. (Marganski & Pancyzk, 2026; Surbhi Kanthed, 2023).

GraphQL API ir API specifikācija un arī vaicājumu valoda, kas nodrošina iespēju precīzi noteikt kādus datus iegūt no sistēmas. Respektīvi, netiek izmantoti galapunkti resursu izgūšanai, bet gan klients pats definē vaicājumu, atbilžu struktūru un datu hierarhiju. Šādā pieejā tiek atsūtīti tikai tie dati, kas ir nepieciešami, nodrošinot to, ka netiek nosūtīti nevajadzīgi dati vai arī iztrūkst dati. (Lawi et al., 2021; Surbhi Kanthed, 2023)

Lai izvēlētos, kura API arhitektūra ir piemērotāka konkrētajā situācijā, jāņem vērā vairāki faktori, piemēram, esošās sistēmas sarežģītība, datu pieprasījumu raksturs, veiktspējas prasības (Surbhi Kanthed, 2023). Pētījumi, kuros tiek salīdzināti RESTful un GraphQL API, norāda, ka ir vairākas atšķirības gan veiktspējā, gan datu pārraides aspektos (Lawi et al., 2021; Marganski & Pancyzk, 2026). Tāpēc ir nepieciešams salīdzināt abus API, lai izvērtētu abu arhitektūru principu galvenās atšķirības jeb faktorus:

- Datu izgūšana - nosaka, kā klients var pieprasīt datus no API un vai ir iespējams precīzi definēt, kādi dati ir nepieciešami vienā pieprasījumā. Faktors ietekmē tīkla pieprasījumu skaitu un datu efektivitāti.
- Mērogojamība - raksturo API spēju saglabāt veiktspēju, kad sistēmai ir paaugstināta slodze. Labs mērogojums norāda, ka sistēma ir stabila pat, ja palielinās datu vai lietotāju apjoms.

- Datu modelis - apraksta, cik strukturēti un loģiski organizēti ir dati API līmenī, kā arī, kā tie ir saistīti savstarpēji. Faktors norāda, cik sarežģīti ir izgūt un apstrādāt saistītus datu elementus.
- Integrācijas vienkāršums - vai API var vienkārši sasaistīt ar esošajām sistēmām. Bez papildus konfigurācijām. Vienkārša integrācija norāda, ka var ieviest ātrāk un ar mazāk kļūdām.
- Klienta sarežģītība - nosaka, cik prasīgs ir API, lietošanai no klienta puses. Zemāka sarežģītība atvieglo izstrādātāja darbu un mazina mācīšanās slieksni.

Zemāk pievienota tabula, kur tiek norādīti definētie faktori un sniegts raksturojums RESTful un GraphQL arhitektūru kontekstā - skatīt pievienoto 1.1. tabulu.

1.1. tabula

RESTful un GraphQL faktoru salīdzinājums

Faktors	RESTful	GraphQL
Datu izgūšanas veids	Klients pieprasa resursus noteiktajos galapunktos	Klients definē vaicājumu un saņem tikai to, kas ir pieprasīts vaicājumā
Mērogojamība	Katrs pieprasījums tiek veikts neatkarīgi un balstās uz definētajiem resursiem	Efektīvi apstrādā sarežģītus datu vaicājumus, taču lielākas slodzes apstākļos nepieciešama papildu optimizācija datu apstrādes mehānismiem
Datu modelis	Balstās uz resursu orientēto pieeju - katram datu tipam atsevišķs galapunkts. Saistītu datu ieguvei nepieciešami vairāki pieprasījumi.	Izmanto shēmā definētu, savstarpēji saistītu datu struktūru - var vienā vaicājumā izgūt vairākus saistītus datu laukus
Integrācijas vienkāršums	Izmanto HTTP-metodes un galapunktus, līdz ar ko nav sarežģīta integrācija	Salīdzinoši elastīgs, taču prasa izpratni par kopēju datu un sistēmas shēmu
Klienta sarežģītība	Klientam ir jāzina konkrētie galapunkti un kā pārvaldīt vairākus pieprasījumus	Sākotnēji var būt sarežģīti - jāpārzina datu un sistēmu shēma

1.1. tabulā apkopotais salīdzinājums norāda, ka starp abām API arhitektūrām pastāv atšķirības ne tikai tehniskajā realizācijā, bet arī datu organizēšanā, pieprasījumu veidošanā un sistēmu integrācijas vienkāršībā. Atšķirības ietekmē gan API izstrādes procesu, gan lietojamību klienta pusē.

1.3. Esošie API dokumentācijas rīki un to ierobežojumi

API dokumentācija nodrošina strukturētu aprakstu par API funkcionalitāti, to izmantošanas nosacījumiem, pieejamajiem resursiem un datu modeļiem. Dokumentācijas pieejamība veicina to, ka gan klienti, gan automatizācijas rīki izprot API darbību bez tiešas piekļuves pirmkodam (*source code*). Tomēr dokumentācijas kvalitāte ietekmē izstrādes procesu efektivitāti - zemākas kvalitātes dokumentācija palielina izstrādes kļūdu skaitu un kavē ātru API apgūšanu jauniem lietotājiem. (Maalej & Robillard, 2013; Meng et al., 2019).

OpenAPI specifikācija ir plaši izmantots standarts, kas definē gan mašīnlasāmu, gan dabīgā valodā saprotamu API apraksta formatējumu, ietverot aprakstītus API ceļus, parametrus un atbildes (*What Is OpenAPI?*, n.d.). Standarta izmantošana veicina strukturētu un konsekventu API dokumentāciju, kas samazina interpretācijas atšķirību risku starp izstrādātājiem, automatizācijas rīkiem un dokumentācijas lietotājiem (*Getting Started*, n.d.). Izmantojot vienotu specifikācijas standartu, API dokumentāciju ir iespējams automatizēt ģenerēt, apstrādāt, vizualizēt, validēt, kas uzlabo dokumentācijas uzturēšanu, kā arī samazina neskaidrību situācijas izstrādes un integrācijas procesos.

1.3.1. API dokumentācijas rīku loģika

Esošie API dokumentācijas rīki izmanto divas galvenās metodes jeb pieejas priekš dokumentācijas ģenerēšanas:

- No koda (*code-first*): sākas ar API implementācijas kodu, kur dokumentācija vai apraksts tiek veidots, analizējot koda komentārus un anotācijas pēc kā tie tiek sasaistīti ar API funkcijām. Risinājums veicina dokumentācijas statusu - sinhronizē ar aktīvo kodu, jo izmaiņas koda struktūrā vai funkcionalitātē var tikt atspoguļotas dokumentācijā. Tomēr izmaiņas ir atkarīgas no komentāru vai anotāciju pilnīguma un kvalitātes, jo, ja izstrādātājs nav detalizēti vai pareizi aprakstījis, tad nebūs precīza dokumentācija (Dudjak & Martinovic, 2020).
- Strukturēta specifikācija (*specification-first*): API dizains un apraksts tiek noteikts pirms programmatūras koda izstrādes, piemēram, izmantojot OpenAPI specifikāciju. Specifikācija tiek izmantota, kā vienots apraksts, ko var izmantot gan dokumentācijas veidošanai, gan

citiem izstrādes posmiem, piemēram, automatizētiem testiem, programmatūras koda izstrādei, noturot konsekventu sapratni par API struktūru visās izstrādes komandās un izmantotajos tehnoloģiskajos rīkos (Chauhan et al., 2025; Yang et al., 2018).

1.3.2. API dokumentācijas rīku piemēri

API dokumentācijas rīku klāts ir ļoti plašs, taču lielākoties praksē izmantotajiem rīkiem ir kopīgas pamatfunkcijas, kas balstītas uz strukturētu API aprakstu formātu izmantošanu (Sundberg et al., 2025). Ņemot vērā plašo rīku klāstu, var detalizētāk apskatīt funkcionalitāti dažiem no zināmākajiem dokumentācijas rīkiem IT nozarē (Elmira, 2024; Kamil, 2026; *Top 10 API Documentation Tools 2025 (with Examples)*, 2024, p. 10). Daži no zināmākajiem dokumentācijas rīkiem:

- *Swagger* - API dokumentācijas ģenerēšanas, vizualizācijas, izpētes rīks, kas ir balstīts uz OpenAPI specifikāciju, ļaujot no JSON vai YAML formātā definētā API apraksta ģenerēt strukturētu, interaktīvu dokumentāciju (*What Is Swagger*, n.d.). Galvenās rīka funkcijas - dokumentācijas vizualizācija, API testēšana, iespēja ģenerēt klienta jeb servera koda skeletus. Rīka galvenā priekšrocība ir tā, ka tas tiek izmantots ar OpenAPI specifikāciju, kas nodrošina saprotamu API struktūru. Tomēr rīks ir orientēts uz OpenAPI specifikācijas vizualizēšanu un interaktīvu API dokumentāciju, un tam nav funkcionalitāte, kas ļautu veidot plašāku tehnisko dokumentāciju arī ārpus API apraksta, kā arī sadarbība iespējas, piemēram, versiju kontrole, ir pieejamas rīka maksas pakalpojumā pieejamas tikai - rīka papildinājums *SwaggerHub (Demir, 2024)*.
- *Redoc* - atvērtā koda rīks API dokumentācijas ģenerāšanai no strukturētas OpenAPI specifikācijas, tostarp JSON vai YAML formatējumā. Sniegto specifikāciju pārveido trīs paneļu dokumentācijā ar pieprasījumu piemēriem, atbildēm un navigāciju starp pieprasījumiem (Redocly, 2015/2026). Priekšrocība šim rīkam ir, ka tas ir viegli uzstādāms, tam ir plašs atbalsts OpenAPI funkciju klāstam (Bush, 2020). Tomēr, rīks nav tik interaktīvs, kā citi rīki, ja netiek veiktas papildus konfigurācijas vai paplašinājumu pievienošana, kā arī

dokumentācijas izvietošana ārpus lokālās izstrādes vides var prasīt izstrādātājiem veikt papildus darbības (Hungbo, 2023).

- *Postman* - API platforma, kas apvieno vairākas API funkcijas vienlaicīgi - dokumentāciju, testēšanu, izstrādi un sadarbības funkcijas. Šādā veidā ļaujot lietotājiem darboties iekš vienas platformas un uzturēt API dokumentāciju vienuviet, kā arī organizēt pieprasījumus kolekcijās, ko var arī koplietot ar citiem lietotājiem (*Postman API Platform - Build, Test & Manage*, n.d.). Galvenās rīka funkcijas ir automātiska dokumentācijas ģenerēšana no definīcijām vai kolekcijām, spēja sinhronizēt dokumentāciju ar API definīciju un spēju rediģēt esošo saturu. Tomēr galvenā priekšrocība ir tas, ka viss ir vienotā platformā pieejams - ne tikai dokumentācija, bet arī testēšana, izstrāde. Bet, lai izmantotu visus iecerētos pakalpojumus var nākties maksāt, kā arī tieši dokumentācijas iespējas var būt mazāk pielāgojamas kā tas ir citviet (*Document Your APIs in Postman | Postman Docs*, n.d.).
- *Stoplight* - API dizaina un dokumentācijas rīku platforma, kas ļauj izstrādātājiem izstrādāt, modelēt, dokumentēt, kā arī publicēt API no OpenAPI specifikācijām. Rīks nodrošina vizuālu API definīciju rediģēšanu, automātisku dokumentācijas ģenerēšanu, kā arī interaktīvu dokumentāciju, kodu piemērus (*OpenAPI Design & Documentation Management Tool | Stoplight*, n.d.; *Solutions for API Design and Management | Stoplight*, n.d.). Var arī veidot publiskus vai privātus API dokumentācijas portālus, kuriem var pielāgot kodu piemērus, navigēšanas dizainu. Priekšrocība ir vienotā platforma, kur gan API dizains, gan dokumentācija ir integrēta vienā vidē, taču, lai pilnībā apgūtu rīka funkcionalitāti būs nepieciešama praktizēties ar to, kā arī pilns funkciju komplekts ir pieejams tikai maksas plānos (*The G2 on Stoplight*, n.d.).

Lai gan API dokumentācijas rīki ievērojami atvieglo strukturētas dokumentācijas gan izveidi, gan uzturēšanu, rīkiem joprojām ir ierobežojumi, kas ir saistīti ar dokumentācijas saturu un kvalitāti. Bieži vien ir sastopami nepilnīgumi, neprecizitātes dokumentācijas saturā, kas būtiski ietekmē API lietošanu (Uddin & Robillard, 2015). API dokumentācijas veidošana un uzturēšana pie apjomīgākām sistēmām prasa

ievērojamus resursus - darbaspēka, laiku, jo ir sarežģīti līdzsvarot dokumentācijas atbilstību API funkcionalitātei vai otrādāk (Bondel et al., 2022). No API dokumentācijas rīku aspekta šie trūkumi nozīmē, ka jebkuras izmaiņas API funkcionalitātē tomēr prasa manuālu dokumentācijas rediģēšanu, jo esošie rīki nespēj atrisināt jaunu risinājumu skaidrojumus bez iesaistes (Uddin & Robillard, 2015).

1.3.3. Ģeneratīvā mākslīgā intelekta bāzēti API dokumentācijas rīki

Pēdējos gados tiek attīstīti tradicionālie dokumentācijas rīku risinājumi implementējot lielos valodas modeļus, lai automatizētu API dokumentācijas vai specifikācijas izveidi. Šādā pieejā tiek izmantots mākslīgais intelekts, lai analizētu programmatūras kodu, nestrukturētus dokumentācijas tekstus, tādejādi automātiski ģenerējot strukturētas API specifikācijas (Deng et al., 2025).

Pētījums "*LRASGen: LLM-based RESTful API Specification Generation"* pētījumā tiek piedāvāts risinājums, kur lielais valodas modelis analizē REST API pirmkodu un automātiski ģenerē OpenAPI specifikāciju, iekļaujot gan API galapunktus, gan parametrus, gan atbilžu struktūras. Pētījuma rezultāti norāda, ka šāda pieeja spēj identificēt vairāk API elementus nekā manuāli izveidotās specifikācijas, tādējādi samazinot manuālās dokumentācijas uzturēšanas apjomu un paralēli palielinot dokumentācijas pilnīgumu (Deng et al., 2025).

Pētījumā "*Automated API Docs Generator using Generator AI*" tiek aprakstīts rīks, kas izmanto lielo valodas modeli, lai no programmatūras koda vai API pieprasījuma struktūras ģenerētu dokumentācijas aprakstus, piemērus un parametrus Šāda veida rīku mērķis ir automatizēt dokumentācijas veidošanu un uzturēšanu, jo tradicionālajās pieejās dokumentācija un faktiskā API funkcionalitāte mēdz nebūt konsekventa (Dhyani et al., 2024).

Paralēli tiek arī izstrādātas sistēmas, kas bāzējas uz lielajiem valodas modeļiem, lai varētu jau no esošas dokumentācijas vai tīmekļa vietnēm automātiski iegūt ģenerētu OpenAPI specifikāciju. Piemēram, *OASBuilder* izmanto lielo valodas modeli un strukturētu datu izgūšanas algoritmu, lai pārveidotu nestrukturētu API dokumentāciju uz mašīnlasāmu specifikāciju, kas pēc tam var tikt izmantota dokumentācijas ģenerēšanai, testēšanai vai automatizētai API integrācijai. Šāda veida risinājumi var optimizēt resursu patēriņu manuālos procesos - dokumentācijas izveidē, pieejamības

uzturēšanā (Lazar, Vetzler, Kate, Tsay, Boaz, et al., 2025; Lazar, Vetzler, Kate, Tsay, Gupta, et al., 2025).

1.4. API dokumentācijas kvalitātes kritēriji

API dokumentācijas kvalitātei ir ievērojama nozīme programmatūras inženierijā. Tā ietekmē, cik ātri un pareizi API lietotāji spēj saprast, integrēt un uzturēt API funkcionalitāti. Lai dokumentācija būtu kvalitatīva ir jādefinē un jāievēro kvalitātes kritēriji (Treude et al., 2020).

API dokumentācijas kvalitāti var raksturot ne tikai no satura aspekta, bet arī no tā, cik labi tiek atbalstītas lietotāja vajadzības un efektīva izmantošana praksē. Tā kā dokumentācija ir starpnieks lietotāja izpratnei un tehniskajai funkcionalitātei, tad tā ir jānovērtē, izmantojot kritērijus, kas atspoguļo tehnisko aspektu un lietojamības aspektu (Cummaudo et al., 2019; Treude et al., 2020). Balstoties uz literatūrā atrastajiem avotiem var spriest, ka šie ir galvenie dokumentācijas kvalitātes kritēriji:

- Precizitāte - dokumentācijai jāsniedz pareiza un nepārprotama informācija par API funkcionalitāti, parametriem, atbildes kodiem, ierobežojumiem. Tādejādi var izvairīties no neskaidrībām vai nepareizas interpretācijas izstrādes laikā (Cummaudo et al., 2019).
- Pilnīgums - dokumentācijai ir jāietver visi API aspekti - metodes, datu struktūras, autentifikācijas noteikumi, kļūdu apstrāde. Šādi tiek nodrošināts, ka lietotājs var atrast visu nepieciešamo informāciju vienā vietā (Cummaudo et al., 2019).
- Saprotamība - dokumentācijas teksta struktūrai, gramatiskumam, terminoloģijai ir jābūt konsekventai un saprotamai, lai gan pieredzējuši, gan jauni izstrādātāji, varētu viegli uztvert būtisko informāciju, bez nepieciešamības prasīt papildus jautājumus interpretācijas kontekstā (Cummaudo et al., 2019; Marinosci & Shchadrynski, 2023).
- Lietojamības atbalsts (praksē) - dokumentācijā ir jāiekļauj praktiski piemēri, piemēram, koda fragmenti, pieprasījuma un atbildes piemēri, īsi lietošanas scenāriji. Šādi piemēri palīdz lietotājam tiešā veidā izprast un testēt API funkcijas, nevis tikai likt lasīt aprakstus (Cummaudo et al., 2019; Marinosci & Shchadrynski, 2023).

- Atjaunošana un aktualitāte - dokumentācijai būtu jābūt sinhronai ar API funkcionalitātes izmaiņām. Ja netiek atjaunota dokumentācija, tad tā var maldināt nākamos dokumentācijas lasītājus un mazināt dokumentācijas lietderību (Cummaudo et al., 2019)
- Piekļuve un organizētība - dokumentācijai ir jābūt viegli pieejamai, organizētai, ar skaidru struktūru, navigēšanas un meklēšanas iespēju. Šādi lietotāji var efektīvi darboties ar dokumentāciju (Johnson, 2022a; Marinosci & Shchadrynski, 2023).

Kopumā kvalitātes kritēriji veido praktisku un strukturētu rāmju kopumu (*framework*), ko var izmantot, lai novērtētu API dokumentāciju. Labi tehniska dokumentācija atvieglos lietotājiem API izpratni, jo tiek samazināta neatbilstība starp dokumentāciju un faktisko funkcionalitāti, kā arī ievērota konkrēta struktūra priekš izpratnes. Šis ir nepieciešams, lai varētu ātri atrast nepieciešamo informāciju par API iespējām vai ierobežojumiem izstrādes procesos (Johnson, 2022b; Meng et al., 2020).

2. MĀKSLĪGĀ INTELEKTA IZMANTOŠNA API ĢENERĒŠANĀ

Mākslīgā intelekta tehnoloģiju, rīku izmantošana programmatūras inženierijā, tieši programmatūras izstrādē un procesu automatizācijā, kļūst par nozares tendenci. Jo jaunu tehnoloģiju ieviešana var ietekmēt darbplūsmas efektivitāti, izstrādes kvalitāti un citas lietas. Pēdējo gadu gan zinātniskie raksti, gan nozares publikācijas norāda, ka ģeneratīvie mākslīgā intelekta rīki ir spējīgi radīt strukturētu atbildi, risinājumu no nestrukturētiem ievad-datiem, piemēram, pārveidojot prasību aprakstus, tekstuālus skaidrojumus par API specifikācijām par formāli definētām API struktūrām vai dokumentāciju. Šādu tehnoloģisku rīku implementēšana veicinātu manuāla darba apjoma samazināšanu un paātrinātu API dokumentācijas sagatavošanu, paralēli nodrošinot arī konsekventumu starp API dokumentāciju un funkcionalitāti, kā arī tiktu ievērota atbilstība esošajiem standartiem (Dhyani et al., 2024).

2.1. Ģeneratīvo modeļu uzbūve un to loma dabīgās valodas pārveidē strukturētā formā

Ģeneratīvie valodas modeļi ir padziļinātas mašīnmācīšanās modeļi, kas spēj prognozēt un ģenerēt strukturētu tekstu, balstoties uz liela apjoma apmācību valodas datu kopām. Modeļu pamatā ir dziļo neironu tīklu arhitektūras, kas ļauj apgūt statistiskās likumsakarības dabīgās valodas lietojumā un veidot vārdu savstarpējās attiecības tekstā (Z. Chen et al., 2024).

Transformatoru arhitektūras ieviešana ievērojami uzlaboja ģeneratīvo modeļu spēju apstrādāt dabīgo valodu, jo ar to palīdzību modeļi var analizēt vārdu savstarpējo attiecību jeb nozīmīgumu no ievadītā tekstu un līdz ar to uztverot var saglabāt kontekstu visa teksta garumā (Maalej & Robillard, 2013). Ievades teksts netiek analizēts secīgi, bet tiek izvērtēts katrs vārds attīecībā pret visiem citiem vārdiem vienlaicīgi, tādejādi nosakot gan tuvākas, gan tālākas vārdu saistības jeb attiecības. Klasiskajās neironu tīklu arhitektūrās vārdi tika analizēti secīgi, tādejādi netika efektīvi uztverts ievades teksta konteksts, ja tiek dotas sarežģītas teksta attiecības (Academy, 2024; Hommel et al., 2022).

Lai māšīnmācīšanās modeļi varētu apstrādāt un interpretēt ievades datus - dabīgas valodas tekstu, simboli un vārdi tiek pārveidoti par skaitliskām vektoru

reprezentācijām, kas satur semantisku informāciju un ļauj modeļiem salīdzināt nozīmes līdzīgumu matemātiskā telpā. Metodi sauc vektoru reprezentācija, kur katrs vārds tiek attēlots kā skaitļu kopa daudzdimensionālā telpā. Vārdi, kuriem ir līdzīga nozīme vai lietojums atrodas blakus vai tuvu viens otram daudzdimensionālajā telpā - līdzīgi vārdi ir arī līdzīgi ciparu jeb skaitļu kopas kontekstā. Šādā veidā modeļi var izprast semantiskās attiecības starp vārdiem un to lietojuma kontekstu, nevis tikai apstrādāt tos kā simbolus, kas līdzšinējās metodēs negarantēja semantiskās līdzības uztveršanu. (Luo & Chong, 2020).

Ģeneratīvie valodas modeļi ir spējīgi pārveidot dabīgās valodas tekstus, aprakstus uz strukturētas formas izvadi. Modeļi ir apmācīti veidot mašīnlasāmu formatējumu izvades, ievērojot noteiktu struktūru, piemēram, JSON vai XML, kas ir viegli lietojami programmatūru un API integrācijās (Shorten et al., 2024). Strukturēta ģenerēšana ir nepieciešama funkcionalitāte mūsdienu modeļos, jo brīva teksta izvadē nav garantija par validitāti un datu lauku hierarhiju, ko nodrošina strukturētas formas izvade (Geng et al., 2025). Strukturētu formu ģenerēšana prasa vairāku metožu pielietojumu, piemēram, modelēšanas shēma, validācijas slāņa izmantošana u.c., jo bez šīm metodēm ģeneratīvie modeļi var izveidot saturiski pareizu atbildi, taču atbilde strukturāli neatbildīs noteiktajai shēmai (Geng et al., 2025; Shorten et al., 2024).

Lai ģeneratīvie valodas modeļi vēl efektīvāk ģenerētu strukturētu izvadi, kas precīzi atbilst vēlamajam formatējumam, tie tiek papildus apmācīti vai pielāgoti konkrētam uzdevumam. Tiek izmantotas metodes, kas veicina modeļa izpratni par formatējuma prasībām un struktūras normām (Lu et al., 2025). Pielāgošana var ietvert modeļa apmācību jeb trenēšanu ar piemēriem, kur ievades tekstam jau atbilst pareizi strukturēts izvades formāts, vai izmantojot shēmu orientētu apmācību - modeļa mērķis ir arī sekot līdzi strukturālajām prasībām, ne tikai ģenerēt izvades saturu (Lu et al., 2025; Z. Shen et al., 2025). Analizējot strukturētu izvades veidošanu, var spriest, ka ir papildus pieejas, metodes, kas veicina shēmas atbilstību un validitāti rezultātā - modeļu apmācība ar papildus slāņiem vai transformācijām. Bez specializētas apmācības vai pielāgošanas modeļi nav tik efektīvi - izvade neatbilst pilnīgai shēmas sintaksei vai validācijas prasībām (Lu et al., 2025; Z. Shen et al., 2025).

2.2. LLM pielietojumi programmatūras inženierijā

Lielie valodas modeļi (*Large Language Models - LLM*) paplašina iespējas automatizācijām programmatūras izstrādes dzīves ciklā. Tas ir pateicoties to spējām gan saprast, gan ģenerēt dabīgā valodā atbildi, gan strukturētu, formatētu atbildi, gan programmatūras kodu. Zinātniskie avoti norāda, ka lielie valodas modeļi var tikt vai jau tiek izmantoti daudzās programmatūras inženierijas uzdevumu jomās - programmatūras izstrādē, programmatūras kvalitātes nodrošināšanā un programmatūras uzturēšanā, protams katrai no jomām ir savi apakš uzdevumi (Hou et al., 2024). Tiek uzskaitīti daži no nozīmīgākajiem programminženierijas jomu uzdevumiem, kur lielie valodas modeļi var tikt vai tiek izmantoti, liekot uzsvaru, ka ievad-dati var būt gan dabīgas valodas teksts, gan koda fragmenti, nevis tikai viens no abiem:

- koda ģenerēšana (*code generation*) - no dabīgās valodas aprakstiem vai esošās dokumentācijas. Šis veicina manuālo kodēšanas apjoma samazināšanu un paātrināt izstrādes ciklu. Lielie valodas modeļi ir spējīgi ģenerēt pilnvērtīgu kodu ar dokumentāciju - komentāriem, tehniskajiem aprakstiem (Dvivedi et al., 2024; Rasheed et al., 2025). Var papildināt, ka atsevišķs apakš uzdevums šim ir koda pabeigšana (*code completion*) - tiek dots iesākts koda fragments ar instrukciju un to ir nepieciešams pabeigt.
- koda pārstrukturēšana (*code refactoring*) - galvenais uzdevums ir uzlabot esošo kodu, veicot daudzas, bet nelielas izmaiņas tā, lai nemainītos koda iepriekšējā funkcionalitāte. Koda pārstrukturēšana var nodrošināt to, ka kods ir vieglāk lasāms, kods tiek atvieglots, kā arī var veicināt jaunu kļūdu vai ievainojamības vietu atrašanu. Koda pārstrukturēšanas gadījumā, esošais kods var būt vieglāk uzturams, kā arī vieglāk uztverams, ja tas netiek izmantots ilgāku laika periodu (Gilis, 2021; Hou et al., 2024).
- koda kopsavilkuma veidošana (*code summarization*) - uzdevums, kura mērķis ir analizēt un izprast esošo kodu, lai varētu automātiski ģenerēt aprakstus no pirmkoda. Šis ir paplašinājums koda dokumentācijai un, ja izdodas veiksmīgi izpildīt, tad ievērojami tiek atvieglota pirmkoda uzturēšana un vēlāka koda fragmentu meklēšana (Hou et al., 2024).

- kļūdu atrašana (*bug localization*) - koda kļūdu (*bug*), kuras nejauši pieļautas programmatūras kodā norādīšana. Kļūdām ir vairāki veidi - sintakses, loģiskās kļūdas un funkcionalitātes kļūdas (Team, 2024). Lielajiem valodas modeļiem šis ir komplekss uzdevums, jo lietotājam ir nepieciešams precīzi definēt ievad-datus - kļūdas ziņojumu vai problēmas analīzes aprakstu, iepriekšējās kļūdas, ja tādas ir, kļūdaino kodu, kas var būt apjomīgs (Hou et al., 2024).
- koda paskaidrošana (*code understanding*) - uzdevums, kā mērķis ir analizēt pirmkodu un saprast tā būtību - struktūru, loģiku, funkcionalitāti, atkarības (D. Shen et al., 2022). Nepieciešams arī saprast izmantoto programmēšanas valodu un izmantotās bibliotēkas, ietvarus (*frameworks*). Lielie valodas modeļi veicot šo uzdevumu var atvieglot lietotāja izpildes procesu, jo dotajiem koda fragmentiem tiek sniegti komentāri vai dokumentācija, kas ļautu lietotājiem ietaupīt gan laiku, gan resursus, kā arī saprast koda būtību (Hou et al., 2024).
- tehniskās dokumentācijas ģenerēšana (*technical documentation generation*) - lielie valodas modeļi ir spējīgi ģenerēt un papildināt jau esošo programmatūras dokumentāciju, ieskaitot API aprakstus, funkciju definīcijas, atbilžu struktūras, lietošanas piemērus. Šāda veida automatizācija paplašina esošo koda kopsavilkumu veidošanu, palīdzot efektīvāk veidot tehniskos artefaktus, ko var izmantot lietotāji (Jorelle, 2024).

Jāatzīmē, ka uzskaitītie pielietojuma piemēri, ir strukturēti un nošķirti uzdevumu veidi, kas ļauj skaidri analizēt lielo valodas modeļu spējas konkrētās programminženierijas jomās vai to uzdevumos. Ikdienā lielie valodas modeļi tiek arī plaši izmantoti kā palīgrīki ikdienas izstrādes situācijās vai spontānos uzdevumos, piemēram, izmantojot rīkus, kā *ChatGPT*, *Claude*, kur lietotāja definētais uzdevums bieži vien ietver kombinācijas no vairāku programminženieriju uzdevumu aspektiem vienlaicīgi - koda ģenerēšana, paskaidrošana. Šādā veidā tiek atspoguļota lielo valodas modeļu potenciāls sniedzot arī risinājumus plašākiem uzdevumiem, kur dota mazāka struktūra kontekstam (Hou et al., 2024).

Darba autors bakalaura darba izstrādes laikā ir veicis pētījumu, kur tika salīdzināta esošo lielo valodas modeļu efektivitāte dažādos programminženierijas

uzdevumos - koda ģenerēšanā, kļūdu atrašana un atkļūdošana. Autors nonāca pie secinājuma, ka modeļus var izmantot kā palīgrīkus, bet nav ieteicams paļauties uz tiem kā pilnvērtīgu gala risinājumu. Iegūtos lielo valodas modeļu uzdevumu risinājumus var apskatīt pievienotajā tīmekļa vietnē: https://github.com/Rihards26/RTU_Bakalaurs_RihardsIevins.git (Ieviņš et al., 2024). Informāciju tehnoloģiju industrija ir dinamiska un strauji attīstās, līdz ar ko iepriekš iegūtie rezultāti var neatspoguļot šī brīža statusu, progresu, tomēr tie dod nozīmīgu ieskatu konkrētajās programmatūras jomās.

2.3. Prompt Engineering un datu konteksta struktūras API ģenerēšanai

Uzvednes inženierija ir mērķtiecīga, sistemātiska dabīgās valodas ievades izstrādes prakse [XXXX]. Prakses mērķis ir veidot strukturētu un kontekstuāli kontrolētas ievades formas, kas satur specifiskas norādes, piemēram, kontekstu un formatējuma prasību, lai varētu maksimāli izmantot lielā valodas modeļa kapacitāti. Šis ir būtiski, jo lielie valodas modeļi pilda uzdoto bāzējoties uz lietotāja prasīto, līdz ar ko - jo precīzāk tiek definēts uzdevums, jo precīzāk, efektīvāk lielais valodas modelis var veikt uzdevuma izpildi (B. Chen et al., 2025; Sahoo et al., 2025).

Prompt struktūras efektivitāte ir atkarīga no tā, kā ir organizēti un definēti atsevišķi elementi jeb komponenti - instrukcija, konteksts, formāta norādīšana, esošā koda fragmenta pievienošana. Dotā uzdevuma instrukcija lielākoties satur uzdevuma aprakstu un prasības, kas virza lielā valodas modeļa darbību uz konkrētu virzienu, taču konteksts un papildus informācijas pievienošana veicina lielā valodas modeļa izpratni par visa uzdevuma specifiku un attiecīgos ierobežojumus (Knoth et al., 2024). Ir pieejami vairāki veidi strukturētiem ievad-datiem, kas var tikt doti valodas modeļiem:

- (*zero-shot learning*) - metode, kad modelim netiek sniegti nekādi piemēri, ir dota tikai uzdevuma instrukcija. Šādā situācijā lielais valodas modelis balstās uz sev pieejamo informāciju jeb savām zināšanām, līdz ar ko šī metode var nesniegt precīzākos rezultātus (*What Is Zero-Shot Prompting?*, 2024)
- (*one-shot learning*) - metode, kad tiek dota uzdevuma instrukcija un tieši viens parauga piemērs, lai ilustrētu uzdevuma struktūru, formatējumu (*Prompt Engineering 101*, n.d.)

- (*few-shot learning*) - metode, kad tiek dota uzdevuma instrukcija un vairāki piemēru kopumi, parasti 3 līdz 5. Šādā veidā lielais valodas modelis spēj saprast vairākas ievades un izvades attiecības, kā arī saprast uzdevuma struktūru. Parauga piemēri tiek izmantoti kā apmācības kopa (*Prompt Engineering 101*, n.d.).

Vēl viena metode - (*fine*-tuning), taču atšķirībā no pārējām metodēm, šajā metodē lielais valodas modelis tiek specifiski papildus apmācīts ar papildus datiem. Šī nav standarta prakses metode, jo tiek mainīti modeļa parametri, bet specifiskos uzdevumos tā var veicināt piemērotāku rezultātu ieguvi.

API specifikāciju ģenerēšana, izmantojot lielos valodu modeļus, pieprasa padziļinātu konteksta strukturēšanu, jo API apraksts jau ir formatēts, strukturēts - ietver noteiktus atribūtus, hierarhijas informāciju. Lai lielais valodas modelis būtu spējīgs precīzi ģenerēt rezultātus, kas atbilst API formātam, ievad-datiem ir jāsatur uzdevuma instrukcija un informācija par API elementiem - galapunktu nosaukumi, izmantotās *HTTP* metodes, atbilžu struktūra utt. Ja ievad-datu konteksts ir nepietiekams vai strukturāli neorganizēts, tad lielais valodas modelis var veidot neskaidrus, nepilnīgus, formāli neatbilstošus rezultātus. Daži pētījumi norāda, ka API ģenerācija ar lielajiem valodas modeļiem ir efektīvāka, ja tiek sniegts papildus strukturēts konteksts, piemēram, esošais kods vai daļēja esošā specifikācija (Deng et al., 2025; Lazar, Vetzler, Kate, Tsay, Gupta, et al., 2025).

Lietotāja ievad-datu strukturēšana priekš lielajiem valodas modeļiem ietekmē rezultāta precizitāti, atkārtojamību un formāta atbilstību ģenerētajos artefaktos. Strukturēta ievade, kur ir definēts uzdevuma konteksts, instrukcija ir nodota ar definētiem mainīgajiem jeb parametriem, marķējumi un formatējuma norādes - lielajam valodas modelim tiek sniegta semantiska un sintaktiska vadlīnija, kas veicina precīzākas izvades jeb rezultātus, piemēram, JSON vai citu strukturētu API specifikāciju (White et al., 2025). Pētījumi liecina, ka strukturēti ievad-dati nodrošina augstāku precizitāti un konsekvenci strukturētu rezultātu ģenerēšanā nekā nestrukturēti ievad-dati. Papildus tiek pieminēts, ka mainīgo jeb parametru nosaukumi ievad-datos arī ietekmē rezultātus (Elnashar et al., 2025).

2.4. Izaicinājumi - semantiskā precizitāte, drošība, versiju kontrole

Lielo valodas modeļu efektīvai izmantošanai programminženierijā ir ievērojami izaicinājumi. Izaicinājumi var ietekmēt gan ģenerētā rezultāta precizitāti, gan drošību, gan atkārtojamības iespēju. Tā kā lielo valodas modeļu darbība ir balstīta uz apmācības kopu datiem, tad pastāv iespējamība, ka ģenerētie rezultāti ne vienmēr atbildīs lietotāja definētajām specifikācijām, tehniskajiem nosacījumiem. Paralēli neatbilstības problēmai ir manipulācijas risks - ļaunprātīgi manipulē ar ievades-datiem, lai iegūtu nevēlamas izvades, kas veicina drošības un kvalitātes riskus. (Alansari & Luqman, 2025; Farquhar et al., 2024). Esošie izaicinājumi ietver sekojošas problēmas:

- semantiskā nekorektība - ģenerētā atbilde neatbilst specifiskajām semantiskajām prasībām - lietotāja definētiem vai iepriekš sistēmas līmenī definētiem. Šādi var veidot kļūmes API artefaktos vai citā strukturētā izvadē (B. Chen et al., 2025; Hou et al., 2024)
- drošības risks - ļaunprātīgi manipulētiem ievad-datiem var iegūt neparedzētu, nevēlamu vai pat bīstamu izvadi, jo lielie valodas modeļi nav spējīgi atšķirt drošus un nedrošus ievades fragmentus (B. Chen et al., 2025; Elnashar et al., 2025; Thota et al., 2026)
- versiju kontroles un atkārtojamības izaicinājumi - lielo valodas modeļu ģenerētās izvades var atšķirties atkarībā no izmantotā modeļa versijas, izmantotajiem apmācības datiem, izmaiņām definētajā *promptā,* kas var veidot izaicinājumus atkārtotu izmantošanu dzīves ciklā un sarežģītu glabāšanas procesu artefaktiem (Z. Chen et al., 2024).

Ir arī vairāki operacionāli un tehniskie ierobežojumi lielajiem valodas modeļiem, kas var būt nozīmīgi API ģenerēšanas un programmatūras inženierijas kontekstā:

- izmaksas - lielo valodas modeļu izmaksas par izmantošanu, kas ietvertu gan apstrādes, gan infrastruktūras izmaksas, varētu būt ievērojamas. Kā arī, ja modelis tiek izmantots intensīvā darbplūsmā ar augstu pieprasījumu skaitu, tad tas arī var palielināt izmaksas.
- Privātuma un datu aizsardzības riski - lielie valodas modeļi var apstrādāt vai glabāt sensitīvu informāciju vai komercnoslēpumus. Līdz ar, ko ir

nepieciešami aizsardzības mehānismi, lai neveidotos datu noplūdes, kas var veidot arī juridiskas sekas (Barbera, 2025)

- Konteksta jeb ievades datu garuma ierobežojumi - esošajiem lielajiem valodas modeļu rīkiem var novērot ievades-datu garuma ierobežojumus. Ierobežojumi var liegt lietotājam sniegt pilnu kontekstu ar paraugiem vai esošo kodu.
- Ārējo servisu pieejamība un savienojuma kvalitāte - lielākoties lielo valodas modeļu pakalpojumi darbojas mākonī bāzētu API veidā, tātad tā pieejamība var būt atkarīga no ārējiem pakalpojumu sniedzējiem.

Nevar nepieminēt cilvēciskā faktora izaicinājumu. Cilvēka ekspertīzes iesaiste ģenerētā risinājuma pārskatīšanā un validēšanā tiek uzskatīta par nozīmīgu procesu. Šādi var sniegt korekcijas jau ievad-datos, nodrošināt papildus kvalitātes kontroli svarīgos uzdevumos, piemēram, API specifikāciju ģenerēšanā u.c (Kathiresan, 2025).

3. AI BALSTĪTAS API SPECIFIKĀCIJAS ĢENERĒŠANAS METODES IZSTRĀDE

Izstrādātā metode tiek balstīta uz mākslīgo intelektu, API specifikāciju ģenerēšanai no sistēmas prasību teksta vai sniegtajiem programmatūras koda fragmentiem. Metodes mērķis ir automatizēt API struktūras identificēšanu un OpenAPI specifikācijas izveidi, izmantojot lielo valodas modeļu spēju gan analizēt programmatūras kodus, gan dabīgās valodas teksta fragmentus. Metode veicinātu manuālā darba apjoma samazināšanos API dokumentācijas veidošanā, kā arī nodrošinātu konsekventumu API struktūras aprakstā (Deng et al., 2025).

Metode balstīta uz vairāku posmu arhitektūru, kur ievadītie lietotāja dati tiek pakāpeniski apstrādāti un transformēti līdz strukturētai API specifikācijai. Metodes darbplūsma sastāv no pieciem galvenajiem komponentiem: ievades datu slānis, priekšapstrādes slānis, mākslīgā intelekta analīzes slānis, specifikācijas ģenerācijas slānis, validācijas slānis. Katrs no šiem komponentiem pilda noteiktu funkciju API struktūras identificēšanas un formalizēšanas procesā.

3.1 Metodes arhitektūras un ievades/izvades datu modelis

Metodes sākuma process ir ievades datu definēšana. Var tikt ievadīti divu veidu dati - sistēmas prasību teksts vai programmatūras koda fragments. Sistēmas prasību teksts ir dabīgā valodā veidots teksts, kas var saturēt informāciju par sistēmas darbību, pieprasījumu veidiem, datu apmaiņas scenārijiem utt. Taču koda fragmenti var ietvert informāciju par funkciju implementācijām vai datu struktūrām. Šāda veida ievades datu pieeja ļautu lietotājam izmantot atšķirīgus informācijas avotus API struktūras identificēšanai.

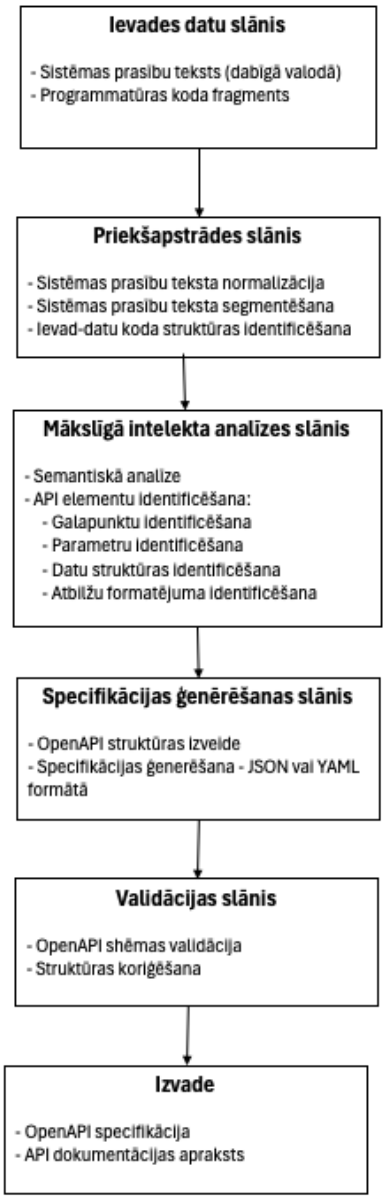
Pēc ievades datu saņemšanas tiek veikta datu priekšapstrāde - datu normalizācija un sagatavošana tālākai analīzei. Dabīgās valodas teksta gadījumā tas ietvertu simbolu un formatējumu normalizāciju, teksta segmentēšanu vai konteksta strukturēšanu. Savukārt koda fragmenta kontekstā tiek identificēti potenciālie API elementi, piemēram, *HTTP* metodes, maršrutēšanas definīcijas vai datu modeļi. Šī posma mērķis ir samazināt nestrukturētās informācijas daudzumu un sagatavot ievad-datus nākamajam posmam - mākslīgā intelekta modeļa analīzei.

Mākslīgā intelekta analīzes posmā tiek analizēti iepriekš transformētie ievades dati. Mākslīgā intelekta modelis analīzē ievadīto dabīgas valodas tekstu vai koda fragmentu un identificē potenciālos API elementus, piemēram, galapunktus, pieprasījuma parametrus, datu un atbildes formātus, struktūras. Posmā tiktu izmantota semantiskā analīze, lai noteiktu saistību starp sistēmas funkcionalitāti un API struktūru. Lielo valodas modeļu spēja interpretēt dabīgās valodas tekstu ļauj pārveidot nestrukturētus prasību aprakstus jau strukturētos API komponentos (Deng et al., 2025).

Kad API elementi ir identificēti, tiek sākts nākamais process - OpenAPI specifikācijas ģenerēšana. Šajā posmā identificētie API komponenti tiktu pārveidoti strukturētā formātā, kas atbilst OpenAPI specifikācijas struktūrai - JSON vai YAML formatējumā. Saturiski tiktu dota informācija par API galapunktiem, *HTTP* metodēm, parametriem, pieprasījumu un atbilžu struktūrām. Strukturēta specifikācija nodrošina vienotu API apraksta formatējumu, ko var izmantot arī dokumentācijas ģenerēšanā, testēšanā vai citos programmatūras izstrādes procesos.

Noslēdzošais metodes posms ir rezultātu validācija. Posmā tiek pārbaudīta iepriekš mākslīgā intelekta ģenerētā specifikācija ar OpenAPI specifikācijas shēmu. Šādā veidā pārliecinoties, ka izveidotais API apraksts atbilst OpenAPI specifikācijas sintakses un struktūras prasībām. Šis posms identificētu trūkumus vai kļūmes - trūkstoši lauki, nepareizi definēti datu tipi, neatbilstoša API struktūra. Validācijas slānis ir nozīmīgs posms, jo ģeneratīvā mākslīgā intelekta modeļi mēdz izvadīt semantiski pareizu atbildi, taču strukturāli neatbilstošu formālajām prasībām.

Zemāk pievienota diagramma, kas ilustrē metodes darbības principu un atspoguļo ievad-datu transformāciju no nestrukturētas informācijas līdz strukturētai API specifikācijai. Skatīt 3.1. attēlu.



3.1. att. Mākslīgā intelekta balstītās API specifikācijas un dokumentācijas ģenerēšanas metodes arhitektūra

Izstrādātās metodes rezultātā tiek ģenerēta OpenAPI specifikācija, kas ietver strukturētu informāciju par API - galapunkti, parametri, datu struktūras. Balstoties uz šo specifikāciju tiek arī izveidots API dokumentācijas apraksts, kas sniegtu cilvēkam saprotamu API funkcionalitātes skaidrojumu un lietošanas piemērus.

3.2. Prasību teksta un koda fragmentu apstrāde

Sākotnēji ir nepieciešams veikt ievades datu apstrādi. Mērķis ir samazināt nestrukturētas informācijas apjomu vienlaikus izceļot potenciāli nozīmīgus API elementus, kas varētu tikt izmantoti nākamajos analīzes procesos. Šis ir būtisks process,

ievades datu apstrāde, jo sistēmas prasību teksti un programmatūras koda fragmenti var būt definēti dažādos veidos, ar atšķirīgu detalizācijas līmeni.

Prasību teksta apstrādes gadījumā tiek izmantotas dabīgās valodas apstrādes (*Natural Language Processing - NLP)* metodes. Dabīgā valoda ir viens no galvenajiem prasību specifikācijas izteiksmes veidiem programminženierijas vidē, bet bieži tā ir nestrukturēta vai neviennozīmīga, kas apgrūtina automatizētu analīzes iespējamību. Dabīgās valodas apstrādes metodes ļauj analizēt šādu tekstu, pārveidojot to strukturētākā formatējumā, kas ir arī atbilstošāka turpmākajiem automatizācijas procesiem (Necula et al., 2024; Zhao et al., 2020).

Prasību dabīgās valodas teksta priekšapstrādes posmā tiek izmantotas vairākas dabīgās valodas apstrādes metodes. Sākotnēji tiek veikta teksta normalizācija un teikumu segmentēšana, kas ļauj sadalīt ievades tekstu atsevišķos teikumos. Pēc kā seko *tokenizācija* - teikumi tiek sadalīti sīkākās vienībās jeb *tokenos*. Nākamais process ir gramatiskā marķēšana (*part-of-speech tagging)* - identificēti darbības vārdi, lietvārdi un citi gramatiskie elementi, kas norāda uz sistēmas darbībām vai datu vienībām. Paralēli gramatiskajai marķēšanai tiek arī veikta sintaktiskā attiecību analīze (*dependency parsing)*, kas ļauj noteikt vārdu nozīmību teikumā (Kolahdouz-Rahimi et al., 2023; Navarro & Ibarra, 2026). Zemāk pievienota tabula ar dabīgās valodas apstrādes metodes soļiem un parauga piemēru, skatīt 3.1. tabulu.

3.1. tabula

Prasību teksta priekšapstrādes posmi un to piemēri

Nr.	Apstrādes solis	Piemērs
1	Teksta normalizācija	"Lietotājs var izveidot jaunu profilu" - noņemti liekie simboli un formatējums no pilna ievades teksta
2	Teikumu segmentēšana	"Lietotājs piesakās sistēmā. Sistēma atgriež lietotāja profilu" - sadalīts divos teikumos
3	Tokenizācija	"Lietotājs izveido profilu" -> [Lietotājs] [izveido] [profilu]
4	Gramatiskā marķēšana	Lietotājs (lietvārds), izveido (darbības vārds), profilu (lietvārds)
5	Sintaktiskā attiecību analīze	Izveido - darbība; profils - objekts
6	Strukturēta reprezentācija	Darbība: izveidot; Objekts: profils

Tomēr ievades-dati var būt arī koda fragmenti, nevis dabīgās valodas teksti. Koda fragmentu apstrāde balstās uz koda struktūras analīzi, kuras laikā tiek identificēti būtiskākie koda elementi, piemēram, funkciju definīcijas, *HTTP* pieprasījumu metodes, datu struktūras u.c. Elementi lielākoties atspoguļo API funkcionalitāti, tādēļ to identificēšana ļautu iegūt būtisku informāciju par iespējamajiem API galapunktiem un to parametriem. Koda fragmentu analīzes rezultātā tiek identificētas potenciālās API operācijas, ievades un izvades parametri un datu struktūras. Šāda strukturēta informācija tiek jau nodota nākamajā metodes posmā.

3.3. API galapunktu, parametru un datu tipu identificēšanas mehānisms

No ievades datu priekšapstrādes tika iegūta strukturēta informācija par sistēmas funkcionalitāti. Līdz ar ko ir iespējams uzsākt nākamo metodes posmu - API galapunktu, pieprasījumu parametru un datu tipu identificēšanu. Posma mērķis ir no iepriekš iegūtās strukturētās informācijas noteikt API struktūras elementus, kas ir nepieciešami OpenAPI specifikācijas izstrādei.

Galapunktu identificēšana balstās uz sistēmas darbības analīzi. Ievades datos jeb prasību tekstā visticamāk sistēmas funkcionalitātes aprakstīšanai tiks izmantoti darbības vārdi, kas apraksta nepieciešamās darbības, piemēram, iegūt, izdzēst, atjaunot. Šāda veida darbības var tikt interpretētas kā REST arhitektūras operācijas, kas atbilst HTTP metodēm - GET, POST, DELETE, PUT. Darbības vārdu semantiskā interpretācija ļauj noteikt gan API galapunkta veidu, gan iespējamo funkcionalitāti.

Galapunktu struktūras veidošanai tiek izmantota arī identificēto objektu analīze, lai attiecinātu izmantotos vai nepieciešamos resursus galapunktam. Piemēram, prasību tekstā var tikt pieminēti objekti kā lietotājs, profils vai pasūtījums, kurus var vēlāk pārveidot attiecīgi par galapunkta identifikatoru - “/lietotajs”, “/profils”, “/pasutijumi”. Respektīvi definētie objekti tiek atspoguļoti API galapunkta nosaukumā. Šādā veidā metode atbilst REST arhitektūras pamatprincipam, kur API tiek veidots kā resursu kopums.

Galapunktu definēšanai ir nepieciešams identificēt arī parametrus. Parametri ir iegūstami ne tikai no dabīgās valodas ievades datiem, bet arī no sniegtajiem koda fragmentiem. Dabīgās valodas aprakstos parametri tiek norādīti kā atribūti vai datu vienības, kas ir nepieciešamas konkrētas darbības izpildei, piemēram, nepieciešams

lietotāja vārds vai parole. Taču koda fragmentos parametri tiek definēti kā funkciju argumenti vai datu struktūru elementi. Analizējot šos elementus ir iespējams iegūt parametru nosaukumus, izmantošanas kontekstu un saistību ar API galapunktu. Parametru identificēšanas procesā tiek analizēta arī to struktūra un nozīme atbilstoši to izmantošanas veidam. Atkarībā no konteksta, parametrus var klasificēt vairākos veidos - ceļa parametri (*path parameters*), pieprasījuma parametri (*query parameters*) vai pieprasījuma ķermeņa (*request body*) parametri jeb dati. Šāda klasifikācija atvieglo OpenAPI specifikācijas izveidi, jo tiek noteikts kādā veidā dati tiek izmantoti API pieprasījumos.

Paralēli parametriem ir nepieciešams noteikt arī to tipus (datu tipus), jo tie raksturo gan pieprasījuma, gan atbildes struktūru. Datu tipu noteikšana balstīta uz semantisko analīzi un konteksta interpretāciju. Koda fragmentos datu tipi lielākoties tiek identificēti tieši, taču dabīgās valodas tekstā tie var tikt identificēti netiešā veidā, tad ir nepieciešams analizēt datu vienību nosaukumus vai to lietojumu kontekstu. OpenAPI specifikācijai ir nepieciešami atbilstoši datu tipi, piemēram, teksta virkne (*string*), vesels skaitlis (*integer*) un citi.

Visi individuāli identificētie API elementi (galapunkti, HTTP metode, parametri, datu tipi) tiek apvienoti strukturētā formatējumā, kas tiek izmantots OpenAPI specifikācijas ģenerēšanai. Lai ilustrētu identificēšanas procesu no prasību teksta, zemāk pievienotajā 3.2. tabulā ir pievienoti īsi piemēri, kas parāda dabīgās valodas prasību teksta aprakstu uz API struktūras elementiem.

3.2. tabula

Prasību teksta transformācija uz strukturētiem API elementiem

Prasību teikums	Galapunkts	HTTP metode	Parametri
Lietotājs izveido profilu	/lietotaji	POST	Vārds, e-pasts, parole
Lietotājs iegūst profilu	/lietotaji/{id}	GET	id - profila identifikators (unikālā vērtība)
Lietotājs atjauno profilu	/lietotaji/{id}	PUT	id, vārds, e-pasts
Lietotājs izdzēš profilu	/lietotaji/{id}	DELETE	id

API elementu identificēšana ļauj automatizēt API izstrādes sākotnējo posmu - plānošanu, projektēšanu. Šādā veidā samazinot manuālā darba apjomu. Šāda pieeja ir

noderīga situācijās, kad API struktūra vēl nav pilnībā definēta vai dokumentācija ir nepilnīga. Rezultātā tiek iegūtu strukturēta, pārskatām informācija, kas tiek izmantota nākamajā metodes posmā - OpenAPI specifikācijas ģenerēšanā un validācijā.

3.4. AI modeļa rezultātu validācija pret OpenAPI shēmu

Pēc API elementu identificēšanas un struktūras izveides tiek ģenerēta OpenAPI specifikācija, taču iegūto specifikāciju ir nepieciešams validēt, ko reprezentē pēdējais metodes posms - rezultātu validācija. Posma mērķis ir pārbaudīt, vai ģenerētā OpenAPI specifikācija atbilst OpenAPI standartam - sintaksei un struktūrai, un vai tā ir vispārīgi izmantojama turpmākajos programmatūras izstrādes procesos.

Ģeneratīvā mākslīgā intelekta modeļi ir spējīgi veidot strukturēti izvadi, tomēr iegūtie rezultāti var būt neprecīzi jeb neatbilstoši formālajām specifikācijas prasībām. Var gadīties situācijas, kad ir trūkstoši obligātie parametri vai nekorekti definēti datu tipi, vai neatbilstoša struktūra. Tādēļ validācijas posms ir būtisks, lai nodrošinātu pēc iespējas augstākas kvalitātes specifikāciju, kura vienlaicīgi atbilstu OpenAPI standartam.

Validācijas process tiek balstīts uz OpenAPI shēmas izmantošanu, kas definē specifikācijas struktūru un obligātos elementus. Procesa laikā tiek pārbaudīti vairāki strukturālie aspekti:

- Verificēta vispārējā struktūra - verificē, ka mākslīgā intelekta ģenerētajā specifikācijā ir iekļauti visi nepieciešamie lauki. Piemēram, *"openapi", "info", "paths"* un citi;
- Pārbaudīta API galapunktu definīcijas - pārbauda HTTP metožu atbilstību, parametru struktūru;
- Datu tipu validācija - tiek validēti datu tipi un to atbilstība OpenAPI specifikācijā definētajiem tipiem.

Papildus strukturālajai validācijai var veikt arī semantisko validāciju, kas pārbauda, vai šie API elementi ir loģiski savstarpēji saistīti. Piemēram, tiek pārbaudīts, vai API galapunktā izmantotie parametri ir definēti un vai šo parametru datu tipi atbilst nepieciešamajai izmantošanai. Šādā veidā var priekšlaicīgi identificēt sintakses kļūdas.

Validācijas procesu, darbības var implementēt ar automatizētiem rīkiem vai programmēšanas valodas bibliotēkām, kas pārbaudītu OpenAPI dokumenta atbilstību

shēmai. Rīki nodrošina kļūdu vai nepilnību identificēšanu, kā arī nodrošina specifikācijas korektumu un lietojamību.

Lai vieglāk pārskatītu validācijas procesu tas tiek ilustrēts ar tabulas palīdzību, sākot ar ievades datiem, pēc kā seko visi veiktie soļi līdz izvadei. Skatīt zemāk pievienoto 3.3. tabulu.

3.3. tabula

Algoritms - OpenAPI specifikācijas validācijas process

Solis/Process	Darbība vai izmantotais resurss
Ievade	Ģenerētā OpenAPI specifikācija
1. Solis	Pārbaudīt OpenAPI specifikācijas vispārējo struktūru
2. Solis	Pārbaudīt vai OpenAPI specifikācija iekļauj obligātos laukus (*openapi, info* un citi)
3. Solis	Validēt API galapunktus un to izmantotās HTTP metodes atbilstību ar uzdoto uzdevumu
4. Solis	Pārbaudīt definētos parametrus un to struktūras
5. Solis	Validēt datu tipus un vai tie atbilst specifikācijai
6. Solis	Identificēt sintakses un struktūras kļūdas
7. Solis	Izlabot identificētās kļūdas vai atgriezt lietotājam turpmākai apstrādei
Izvade	Validēta un atbilstoša OpenAPI specifikācija

Gadījumos, kad kādā no validācijas procesa soļiem tiek atrasta kļūda, tā tiek novērsta atkarībā no kļūdas veida. Zemāk pievienota tabula ar dažiem kļūdu paraugiem un to potenciālajiem risinājumiem, skatīt 3.4. tabulu.

3.4. tabula

OpenAPI specifikācijas validācijas kļūdu piemēri un to novēršana

Kļūdas veids	Piemērs	Risinājums
Specifikācijā trūkst lauks "*paths*"	Nav definēti galapunkti	Pievienot "*paths*" sadaļu specifikācijā
Nepareizs datu tips	ID = *string*, bet jābūt *integer*	Izlabot tipu
Trūkst parametra definīcija	Izmantots ID, bet tas nav definēts	Pievienot jaunu parametru

Validācijas rezultātā tiek iegūta standartam atbilstoša OpenAPI specifikācija, kas ir arī validēta, samazinot semantiskās un tehniskās kļūdas. Specifikāciju var izmantot API dokumentācijas ģenerēšanai vai integrācijai citos programmatūras risinājumos.

4. PROTOTIPA IZSTRĀDE UN VALIDĀCIJA

4.1. Izmantotā tehnoloģiju kopa

4.2. Prototipa funkcionalitāte

4.3. Testa scenāriji un datu kopas

4.4. Lietotāju testēšana un rezultātu novērtējums

5. AI ĢENERĒTĀS UN CILVĒKA VEIDOTĀS DOKUMENTĀCIJAS SALĪDZINOŠĀ ANALĪZE

5.1. Novērtēšanas kritēriji

5.2. Rezultāti un to analīze

5.3. Trūkumi un uzlabojumu iespējas

REZULTĀTI UN SECINĀJUMI

IZMANTOTIE INFORMĀCIJAS AVOTI

Academy, E. (2024, June 11). Kā transformatoru modeļos esošais sevis uzmanības mehānisms uzlabo liela attāluma atkarību apstrādi dabiskās valodas apstrādes uzdevumos? - EITCA akadēmija. *EITCA Academy*. https://lv.eitca.org/m%C4%81ksl%C4%ABgais-intelekts/eitc-ai-adl-padzi%C4%BCin%C4%81ta-m%C4%81c%C4%AB%C5%A1an%C4%81s/dabisk%C4%81s-valodas-apstr%C4%81de/padzi%C4%BCin%C4%81ta-m%C4%81c%C4%AB%C5%A1an%C4%81s-dabisk%C4%81s-valodas-apstr%C4%81dei/eks%C4%81menu-p%C4%81rskats-uzlabotas-dzi%C4%BC%C4%81s-m%C4%81c%C4%ABbas-dabisk%C4%81s-valodas-apstr%C4%81dei/k%C4%81-transformatoru-mode%C4%BCos-eso%C5%A1ais-sevis-uzman%C4%ABbas-meh%C4%81nisms-uzlabo-liela-att%C4%81luma-atkar%C4%ABbu-apstr%C4%81di-dabisk%C4%81s-valodas-apstr%C4%81des-uzdevumos/

Alansari, A., & Luqman, H. (2025). *Large Language Models Hallucination: A Comprehensive Survey* (arXiv:2510.06265). arXiv. https://doi.org/10.48550/arXiv.2510.06265

Barbera, I. (2025, February). *AI Privacy Risks & Mitigations Large Language Models (LLMs)*. https://www.edpb.europa.eu/system/files/2025-04/ai-privacy-risks-and-mitigations-in-llms.pdf

Boateng, R., Ofoeda, J., & Effah, J. (2019). Application Programming Interface (API) Research: A Review of the Past to Inform the Future. *Int. J. Enterp. Inf. Syst.*, *15*(3), 76–95. https://doi.org/10.4018/IJEIS.2019070105

Bondel, G., Cerit, A., & Matthes, F. (2022). Challenges of API Documentation from a Provider Perspective and Best Practices for Examples in Public Web API Documentation: *Proceedings of the 24th International Conference on Enterprise Information Systems*, 268–279. https://doi.org/10.5220/0011089700003179

Bush, T. (2020, June 30). Review of Redoc: Instant API Documentation | Nordic APIs |. *Nordic APIs*. https://nordicapis.com/review-of-redoc-instant-api-documentation/

Casas, S., Cruz, D., Vidal, G., & Constanzo, M. (2021). *Uses and applications of the OpenAPI/Swagger specification: A systematic mapping of the literature*. 1–8. https://doi.org/10.1109/SCCC54552.2021.9650408

Chauhan, S., Rasheed, Z., Sami, M. A., Kemell, K.-K., Waseem, M., Zhang, Z., Rasku, J., Saari, M., & Abrahamsson, P. (2025). *From Specification to Service: Accelerating API-First Development Using Multi-Agent Systems* (arXiv:2510.19274). arXiv. https://doi.org/10.48550/arXiv.2510.19274

Chen, B., Zhang, Z., Langrené, N., & Zhu, S. (2025). Unleashing the potential of prompt engineering for large language models. *Patterns*, *6*(6), 101260. https://doi.org/10.1016/j.patter.2025.101260

Chen, Z., Xu, L., Zheng, H., Luyao, C., Tolba, A., Zhao, L., Yu, K., & Feng, H. (2024). Evolution and Prospects of Foundation Models: From Large Language Models to Large Multimodal Models. *Computers, Materials and Continua*, *80*(2), 1753–1808. https://doi.org/10.32604/cmc.2024.052618

Cummaudo, A., Vasa, R., & Grundy, J. (2019). *What should I document? A preliminary systematic mapping study into API documentation knowledge* (arXiv:1907.13260). arXiv. https://doi.org/10.48550/arXiv.1907.13260

Decrop, A., Devroey, X., Papadakis, M., Schobbens, P.-Y., & Perrouin, G. (2025). *You Can REST Now: Automated REST API Documentation and Testing via LLM-Assisted Request Mutations* (arXiv:2402.05102). arXiv. https://doi.org/10.48550/arXiv.2402.05102

Demir, D. (2024, January 30). *Swagger vs SwaggerHub: Making the Right Choice for API Development*. https://apidog.com/blog/swagger-vs-swaggerhub/

Deng, S., Huang, R., Zhang, M., Cui, C., Towey, D., & Wang, R. (2025). *LRASGen: LLM-based RESTful API Specification Generation* (arXiv:2504.16833). arXiv. https://doi.org/10.48550/arXiv.2504.16833

Dhyani, P., Nautiyal, S., Negi, A., Dhyani, S., & Chaudhary, P. (2024). Automated API Docs Generator using Generative AI. *2024 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, 1–6. https://doi.org/10.1109/SCEECS61402.2024.10482119

Document your APIs in Postman | Postman Docs. (n.d.). Retrieved February 22, 2026, from https://learning.postman.com/docs/publishing-your-api/api-documentation-overview

Dudjak, M., & Martinovic, G. (2020). An API-first methodology for designing a microservice-based Backend as a Service platform. *Information Technology and Control*, *49*, 206–223. https://doi.org/10.5755/j01.itc.49.2.23757

Dvivedi, S. S., Vijay, V., Pujari, S. L. R., Lodh, S., & Kumar, D. (2024). *A Comparative Analysis of Large Language Models for Code Documentation Generation* (arXiv:2312.10349). arXiv. https://doi.org/10.48550/arXiv.2312.10349

Elmira. (2024, May 1). 12 Best API Documentation Software and Tools for 2026. *ClickHelp*. https://clickhelp.com/clickhelp-technical-writing-blog/12-best-api-documentation-software-and-tools/

Elnashar, A., White, J., & Schmidt, D. C. (2025). Enhancing structured data generation with GPT-4o evaluating prompt efficiency across prompt styles. *Frontiers in Artificial Intelligence*, *8*, 1558938. https://doi.org/10.3389/frai.2025.1558938

Farquhar, S., Kossen, J., Kuhn, L., & Gal, Y. (2024). Detecting hallucinations in large language models using semantic entropy. *Nature*, *630*(8017), 625–630. https://doi.org/10.1038/s41586-024-07421-0

Geng, S., Cooper, H., Moskal, M., & Jenkins, S. (2025, January). *(PDF) Generating Structured Outputs from Language Models: Benchmark and Studies*. ResearchGate. https://doi.org/10.48550/arXiv.2501.10868

Getting Started. (n.d.). OpenAPI Documentation. Retrieved February 22, 2026, from https://learn.openapis.org/

Gilis, A. S. (2021, September 15). *What is Refactoring (Code Refactoring)?* Search App Architecture. https://www.techtarget.com/searchapparchitecture/definition/refactoring

Hommel, B. E., Wollang, F.-J. M., Kotova, V., Zacher, H., & Schmukle, S. C. (2022). Transformer-Based Deep Neural Language Modeling for Construct-Specific Automatic Item Generation. *Psychometrika*, *87*(2), 749–772. https://doi.org/10.1007/s11336-021-09823-9

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). *Large Language Models for Software Engineering: A Systematic Literature Review* (arXiv:2308.10620). arXiv. https://doi.org/10.48550/arXiv.2308.10620

Hungbo, M. (2023, March 2). *A Guide to Creating API Documentation With Redoc and GitHub Pages*. DEV Community. https://dev.to/heymich/a-guide-to-creating-api-documentation-with-redoc-and-github-pages-4i4a

Ieviņš, R., Karlovs-Karlovskis, U., & Šāblis, A. (2024). *Radošā mākslīgā intelekta spēju analīze programminženierijas vajadzībām*. https://ndr.rtu.lv/lv/view/39616/

Irum Raufa, Elena Troubitsynab, & Ivan Porresc. (2019). *A Systematic Mapping Study of API Usability Evaluation Methods*. https://oro.open.ac.uk/61584/1/article-2019.pdf

Johnson, T. (2022a, February). *Different approaches for assessing information quality*. I'd Rather Be Writing Blog and API Doc Course. https://idratherbewriting.com/learnapidoc/docapis_metrics_assessing_information_quality.html

Johnson, T. (2022b, February). *Quality checklist for API documentation*. I'd Rather Be Writing Blog and API Doc Course. https://idratherbewriting.com/learnapidoc/docapis_quality_checklist.html

Jorelle, Y. (2024). *Generation of API Documentation using Large Language Models—Towards Self-explaining APIs*. https://aaltodoc.aalto.fi/server/api/core/bitstreams/b56655a0-32c3-4c19-a816-5bddebf01a6e/content

Kamil, I. (2026, January 12). *Top API Documentation Tools in 2026: The Shortlist for Modern API Teams*. DEV Community. https://dev.to/arobakid/top-api-documentation-tools-in-2026-the-shortlist-for-modern-api-teams-29bj

Karlsson, S., Causevic, A., & Sundmark, D. (2019). *QuickREST: Property-based Test Generation of OpenAPI-Described RESTful APIs* (arXiv:1912.09686). arXiv. https://doi.org/10.48550/arXiv.1912.09686

Kathiresan, G. (2025, May). *(PDF) Human-in-the-Loop Testing for LLM-Integrated Software: A Quality Engineering Framework for Trust and Safety*. ResearchGate. https://doi.org/10.36227/techrxiv.174702077.78864934/v1

Knoth, N., Tolzin, A., Janson, A., & Leimeister, J. M. (2024). AI literacy and its implications for prompt engineering strategies. *Computers and Education: Artificial Intelligence*, *6*, 100225. https://doi.org/10.1016/j.caeai.2024.100225

Kolahdouz-Rahimi, S., Lano, K., & Lin, C. (2023). *Requirement Formalisation using Natural Language Processing and Machine Learning: A Systematic Review* (arXiv:2303.13365). arXiv. https://doi.org/10.48550/arXiv.2303.13365

Lamothe, M., Guéhéneuc, Y.-G., & Shang, W. (2021). A Systematic Review of API Evolution Literature. *ACM Comput. Surv.*, *54*(8), 171:1-171:36. https://doi.org/10.1145/3470133

Lawi, A., Panggabean, B. L. E., & Yoshida, T. (2021). Evaluating GraphQL and REST API Services Performance in a Massive and Intensive Accessible Information System. *Computers*, *10*(11), 138. https://doi.org/10.3390/computers10110138

Lazar, K., Vetzler, M., Kate, K., Tsay, J., Boaz, D., Gupta, H., Shinnar, A., Vallam, R. D., Amid, D., Goldbraich, E., Laredo, J., & Anaby Tavor, A. (2025). Generating OpenAPI Specifications from Online API Documentation with Large Language Models. In G. Rehm & Y. Li (Eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)* (pp. 237–253). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-industry.18

Lazar, K., Vetzler, M., Kate, K., Tsay, J., Gupta, D. B. H., Shinnar, A., Vallam, R. D., Goldbraich, D. A. E., Uziel, G., Laredo, J., & Tavor, A. A. (2025). *OASBuilder: Generating OpenAPI Specifications from Online API Documentation with*

Large Language Models (arXiv:2507.05316). arXiv. https://doi.org/10.48550/arXiv.2507.05316

Lu, Y., Li, H., Cong, X., Zhang, Z., Wu, Y., Lin, Y., Liu, Z., Liu, F., & Sun, M. (2025). *Learning to Generate Structured Output with Schema Reinforcement Learning* (arXiv:2502.18878). arXiv. https://doi.org/10.48550/arXiv.2502.18878

Luo, J. W., & Chong, J. J. R. (2020). Review of Natural Language Processing in Radiology. *Neuroimaging Clinics of North America, Machine Learning and Other Artificial Intelligence Applications*, *30*(4), 447–458. https://doi.org/10.1016/j.nic.2020.08.001

Maalej, W., & Robillard, M. P. (2013). Patterns of Knowledge in API Reference Documentation. *IEEE Transactions on Software Engineering*, *39*(9), 1264–1282. https://doi.org/10.1109/TSE.2013.12

Marganski, P., & Pancyzk, B. (2026). (PDF) REST and GraphQL comparative analysis. *ResearchGate*. https://doi.org/10.35784/jcsi.2473

Marinosci, G., & Shchadrynski, R. (2023). *Improving the accessibility of API documentation for non-technical users*. https://urn.kb.se/resolve?urn=urn:nbn:se:lnu:diva-121828

Meng, M., Steinhardt, S. M., & Schubert, A. (2020, October). (PDF) Optimizing API Documentation: Some Guidelines and Effects. *ResearchGate*. https://doi.org/10.1145/3380851.3416759

Meng, M., Steinhardt, S., & Schubert, A. (2019). (PDF) How developers use API documentation: An observation study. *ResearchGate*. https://doi.org/10.1145/3358931.3358937

Navarro, J., & Ibarra, R. (2026). Automatic test case generation using natural language processing: A systematic mapping study. *Information and Software Technology*, *189*, 107929. https://doi.org/10.1016/j.infsof.2025.107929

Necula, S.-C., Dumitriu, F., & Greavu-Șerban, V. (2024). A Systematic Literature Review on Using Natural Language Processing in Software Requirements Engineering. *Electronics*, *13*(11), 2055. https://doi.org/10.3390/electronics13112055

OpenAPI Design & Documentation Management Tool | Stoplight. (n.d.). Retrieved February 22, 2026, from https://stoplight.io/

OpenAPI Specification v3.2.0. (n.d.). Retrieved January 26, 2026, from https://spec.openapis.org/oas/v3.2.0.html

Postman API Platform—Build, Test & Manage. (n.d.). Retrieved February 22, 2026, from https://www.postman.com/product/

Prompt Engineering 101: Understanding Zero-Shot, One-Shot, and Few-Shot. (n.d.). Codecademy. Retrieved February 28, 2026, from https://www.codecademy.com/article/prompt-engineering-101-understanding-zero-shot-one-shot-and-few-shot

Rasheed, Z., Waseem, M., Kemell, K.-K., & Ahmad, A. (2025, January). *(PDF) Large Language Models for Code Generation: The Practitioners Perspective*. ResearchGate. https://doi.org/10.48550/arXiv.2501.16998

Redocly. (2026). *Redocly/redoc* [TypeScript]. https://github.com/Redocly/redoc (Original work published 2015)

Sahoo, P., Singh, A. K., Saha, S., Jain, V., Mondal, S., & Chadha, A. (2025). *A Systematic Survey of Prompt Engineering in Large Language Models:*

Techniques and Applications (arXiv:2402.07927). arXiv. https://doi.org/10.48550/arXiv.2402.07927

Shen, D., Chen, X., Wang, C., Sen, K., & Song, D. (2022). *Benchmarking Language Models for Code Syntax Understanding* (arXiv:2210.14473; Version 1). arXiv. https://doi.org/10.48550/arXiv.2210.14473

Shen, Z., Wang, D. Y.-B., Mishra, S. S., Xu, Z., Teng, Y., & Ding, H. (2025). SLOT: Structuring the Output of Large Language Models. In S. Potdar, L. Rojas-Barahona, & S. Montella (Eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track* (pp. 472–491). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.emnlp-industry.32

Shorten, C., Pierse, C., Smith, T. B., Cardenas, E., Sharma, A., Trengrove, J., & Luijt, B. van. (2024). *StructuredRAG: JSON Response Formatting with Large Language Models* (arXiv:2408.11061). arXiv. https://doi.org/10.48550/arXiv.2408.11061

Solutions for API Design and Management | Stoplight. (n.d.). Retrieved February 22, 2026, from https://stoplight-site.webflow.io/solutions

Sundberg, E., Ekmark, T., & Workneh, A. Y. (2025). *Validating API Design Requirements for Interoperability: A Static Analysis Approach Using OpenAPI*. Zenodo. https://doi.org/10.5281/ZENODO.17447913

Surbhi Kanthed. (2023). (PDF) Rest vs. GraphQL: Comparative Analysis of API Design Approaches. *ResearchGate*. https://doi.org/10.54660/.IJMRGE.2023.4.1.984-991

Team, Q. (2024, January 3). Understanding the Distinction Between Code Bugs and Defects in Software Testing. *Qodo*. https://www.qodo.ai/blog/understanding-the-distinction-between-code-bugs-and-defects-in-software-testing/

The G2 on Stoplight. (n.d.). G2. Retrieved February 22, 2026, from https://www.g2.com/products/stoplight/reviews

Thota, Y. R., Rafatirad, S., Houman, H., & Nikoubin, T. (2026). *When Models Ignore Definitions: Measuring Semantic Override Hallucinations in LLM Reasoning* (arXiv:2602.17520). arXiv. https://doi.org/10.48550/arXiv.2602.17520

Top 10 API Documentation Tools 2025 (with examples). (2024, December 30). DEV Community. https://dev.to/ismailkamil/top-10-api-documentation-tools-in-2025-with-doc-examples-3pe4

Treude, C., Middleton, J., & Atapattu, T. (2020). Beyond accuracy: Assessing software documentation quality. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020*, 1509–1512. https://doi.org/10.1145/3368089.3417045

Uddin, G., & Robillard, M. P. (2015). (PDF) How API Documentation Fails. *ResearchGate*. https://doi.org/10.1109/MS.2014.80

What is OpenAPI? – OpenAPI Initiative. (n.d.). Retrieved February 22, 2026, from https://www.openapis.org/what-is-openapi

What is Swagger. (n.d.). Swagger Docs. Retrieved February 22, 2026, from https://swagger.io/docs/specification/v2_0/what-is-swagger/

What is zero-shot prompting? | IBM. (2024, September 10). https://www.ibm.com/think/topics/zero-shot-prompting

White, J., Elnashar, A., & Schmidt, D. (2025, June 24). *Prompt Engineering for Structured Data A Comparative Evaluation of Styles and LLM Performance[v1] | Preprints.org*. https://www.preprints.org/manuscript/202506.1937

Yang, J., Wittern, E., Ying, A. T. T., Dolby, J., & Tan, L. (2018). *Automatically Extracting Web API Specifications from HTML Documentation* (arXiv:1801.08928). arXiv. https://doi.org/10.48550/arXiv.1801.08928

Zhao, L., Alhoshan, W., Ferrari, A., Letsholo, K. J., Ajagbe, M. A., Chioasca, E.-V., & Batista-Navarro, R. T. (2020). *Natural Language Processing (NLP) for Requirements Engineering: A Systematic Mapping Study* (arXiv:2004.01099). arXiv. https://doi.org/10.48550/arXiv.2004.01099

PIELIKUMI

1. pielikums

Pielikuma nosaukums

Qwerty qwerty qwerty qwerty qwerty. Asdfg asdfg asdfg asdfg asdfg asdfg. Qwerty qwerty qwerty qwerty qwerty. Asdfg asdfg asdfg asdfg asdfg asdfg. Qwerty qwerty qwerty qwerty qwerty.